

Notes for Nested Loops (Loops Inside Loop)

(07/10/2023, Subhasis Das)

P-1

Structure of Nested Loops:

```
Loop(.....){
    Loop(.....){
    }
}
```

Example 1 (for inside for)

```
for(int i=1; i<n ; i++){
    for(int j=1; j<n; j++){
        //some code
    }
}
```

Example 2(while inside while)

```
int i=1;
while(i<=n){
    int j=1;
    while(j<=n){
        //some code
    }
}
```

Example 3 (while inside for)

```
for(int i=1; i<=n; i++){
    int j=1;
    while(j<=n){
        //some code
    }
}
```

****Similarly, for inside while, while inside do-while, do-while inside while etc can also be made**

Property: If Outer loop runs for **n** times and inner loop runs for **m** times, Total number of runs will be **nm**

Example :

```
for(int i=1; i<=3; i++){
    for(int j=1; j<=4; j++){
        System.out.println("HI");
    }
}
```

**** Here the outer loop is running 3 times and inner loop is running 4 times, then total run will be $3 \times 4 = 12$ times**

**** For each iteration of outer loop, the inner loops complete all its iterations. (if not terminated using jump statements)**

Output: HI HI HI HI HI HI HI HI HI HI HI HI

Dependent and Independent Loops:

Dependent Loops: -

****If the inner loop is depending on the control variable of outer loop, it's a dependent loop.**

Example 1

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        //some code
    }
}
```

Example 2

```
for(int i=1; i<=5; i++){
    for(int j=i; j<=5; j++){
        //some code
    }
}
```

Example 3

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=5; j+=i){
        //some code
    }
}
```

Independent Loops:

P-2

If the inner loop is running for some fixed number of times, it's an **independent loop.

Example 1

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=4; j++){

        //some code
    }
}
```

Example 2

```
for(int i=1; i<=5; i++){
    for(int j=2; j<=5; j++){

        //some code
    }
}
```

Example 3

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=5; j+=3){

        //some code
    }
}
```

** As you can see in the above codes, both outer and inner loops run for fixed number of times.

Labelling Of Loops

** We can add labels to loops. Generally, this is used in nested loops to use jump statements more specifically. For example, If you want to **break** or **continue** outer loop from inner loop.

Syntax:

Label:

```
for(int i=1; i<=10; i++){
    //some code
}
```

Example:

MyLoop:

```
for(int i=1; i<=10; i++){
    System.out.println(i);
}
```

Use in Nested Loops:

Example 1:

outerloop:

```
for (int i = 1; i <= 5; i++) {
    innerloop:
        for (int j = 1; j <=5; j++) {
            if (j==5) {
                break outerloop;
            }
        }
}
```

Output 1:

1234

Example 2:

outerloop:

```
for (int i = 1; i <= 5; i++) {
    innerloop:
        for (int j = 1; j <=5; j++) {
            if (j==5) {
                continue outerloop;
            }
        }
}
```

Output 2:

12341234123412341234

** Using break or continue statement without label will affect the loop inside which the statement is used.

Dependent Loop Symbol Patterns:

Step 1: Count the number of symbols in each line , write it.

Step 2: The series you got after counting, make a loop out of it. (assuming control variable is i)

Step 3: as the inner loop is printing i number of symbols, write a loop that runs for i times . (assuming j is the control variable)

Step 4: Print the symbol without changing line

Step 5: change line in outerloop.

Example 1:

```
* ----- 1
* * ----- 2
* * * ----- 3
* * * * ----- 4
* * * * * ----- 5
```

Code:

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        System.out.print("*");
    }
    System.out.println();
}
```

Example 2:

```
* ----- 1
* * * ----- 3
* * * * ----- 5
* * * * * ----- 7
```

Code:

```
for(int i=1; i<=7; i+=2){
    for(int j=1; j<=i; j++){
        System.out.print("*");
    }
    System.out.println();
}
```

Example 3: (Symbol depends on j value)

```
*
* #
* # *
* # * #
* # * # *
```

Code:

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        if (j%2==0) System.out.print("#");
        else System.out.print("*");
    }
    System.out.println();
}
```

Example 3.1 (Symbol depends on i value)

```
*
# #
* * *
# # # #
* * * * *
```

Code:

```
for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        if (i%2==0)
            System.out.print("#");
        else System.out.print("*");
    }
    System.out.println();
}
```

** In example 3, * or # depends on whether j is odd or even, similarly in example 3.1, it depends on whether i is odd or even

Independent Loop Symbol Patterns

Step 1: Count the number of rows (R).

Step 2: Run Outer loop (i) for R number of times.

Step 3: Count the number of columns (C)

Step 4: Run Inner Loop (j) for C number of times.

Step5: Print Symbol without line change

Step 6: Change line in Outerloop.

Example 1:

```

* * * * *
* * * * *
* * * * *
* * * * *
    
```

R(Rows)=4
C(Columns)=5

Code :

```

for(int i=1; i<=4; i++){
    for(int j=1; j<=5; j++){
        System.out.print("*");
    }
    System.out.println();
}
    
```

Number Patterns

Dependent Loop "Number" Patterns:

Type 1: One Side Constant, One Side Variable

Step 1: Mark the sides. One side should be constant and another should be variable. Mark the constant side with the constant, Mark the variable side with **i**

Step 2: Define **i** with a loop (Outer Loop)

Step 3: The Inner Loop (j) will run from left side to right side as marked.

Step 4: Print j inside inner loop without any line change

Step 5: Change line in outerloop

Example 1 (Right side variable):

Code:

```

for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        System.out.print(j);
    }
    System.out.println();
}
    
```

Example 2 (Left Side Variable):

Code:

```

for(int i=5; i>=1; i--){
    for(int j=i; j>=1; j--){
        System.out.print(j);
    }
    System.out.println();
}
    
```

Type 2 (Both Side Variable):

Step 1: Mark both the sides with **i** and **k**

Step 2: Write outer loop with both the variables (**i** and **k**) (parallel)

Step 3: Write inner loop (j), which runs from Left side to Right side

Step 4: Print j inside inner loop without changing line

Step 5: Change the line in outer loop

Example 1 :

p-5

Code:

```
for(int i=1, k=1; i<=4; i++, k+=2){
    for(int j=i; j<=k; j++){
        System.out.print(j);
    }
    System.out.println();
}
```

Type 3 : Printing Same elements

Step 1: Design the given shape like Symbol Patterns

Step 2: Initialise a third variable(n) with starting point.

Step 3: Print n inside inner loop without changing line.

Step 4: Update value of n in **outer** loop. (as the line changes the value also changes)

Example 1:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Code:

```
int n=1;
for(int i=1; i<=5; i++){
    for(int j=1; j<=i; j++){
        System.out.print(n);
    }
    System.out.println();
    n++;
}
```

Type 4: Printing Consecutive Elements

Step 1: Design the given shape like Symbol Patterns

Step 2: Initialise a third variable(n) with starting point.

Step 3: Print n inside inner loop without changing line.

Step 4: Update value of n in **inner** loop. (as the line changes the value also changes)

Example 1:

```
1
2 3
4 5 6
7 8 9 10
```

Code :

```
int n=1;
for(int i=1; i<=4; i++){
    for(int j=1; j<=i; j++){
        System.out.print(n);
        n++;
    }
    System.out.println();
}
```

Independent Loop "Number" Patterns

p-6

Type 1 (Same Series in each line)

Step 1: Find number of Rows (R)

Step 2: Write Outer loop (i) which runs for R times

Step 3: Write the inner loop (j) for the series which is getting printed on each line.

Step 4: Print j without changing line

Step 5: change the line in outer loop

1 2 3 4 5 (Rows (R) = 4, in each line 12345 is getting printed)
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

Code:

```
for(int i=1; i<=4; i++){  
    for(int j=1; j<=5; j++){  
        System.out.print(j);  
    }  
    System.out.println();  
}
```

Type 2: Same elements in each line

Step 1: Find number of Rows (R) and Columns (C)

Step 2: Write Outer loop (i) which runs for R times

Step 3: Write the inner loop (j) for C times

Step 4: Initialise a third variable (n) with starting point

Step 5: Print n inside inner loop without changing line

Step 5: update n in outer loop.

Step 6: Change the line in outer loop

1 1 1 1 1 Rows(R)= 5, Columns (C)=5
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
5 5 5 5 5

Code:

```
int n=1;  
for(int i=1; i<=5; i++){  
    for(int j=1; j<=5; j++){  
        System.out.print(n);  
    }  
    System.out.println();  
    n++;  
}
```

Another Approach:

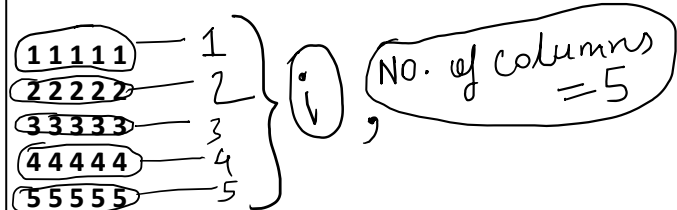
Step 1 : Observe what is being printed on each line, Make a series out of it, Write the outerloop (i) for it.

Step 2: Count the number of Columns (c), and Run Inner Loop (j) for C times.

Step 3 : Print i in Inner loop without changing line

Step 4: Change Line in Outer loop

Example 1:



Code:

```
for(int i=1; i<=5; i++){  
    for(int j=1; j<=5; j++){  
        System.out.print(i);  
    }  
    System.out.println();  
    n++;  
}
```

Sum Of Series (Nested)

p-7

**We need to understand two symbols in order to gain better understanding of sum of series.

Capital Sigma (Σ) and Capital Pi (Π)

Sigma (Summation)

Σ This symbol is used to represent summation. There are two types of it – Limited and Limitless. We are interested in limited Sigma for our purpose.

The limited Sigma has two limits, upper limit and lower limit and a function to find sum of.

represented as $\sum_{\text{lower limit}}^{\text{Upper limit}} f$, It finds sum of all f from lower limit to upper limit with an update of +1

Example 1:

$$\sum_{i=1}^n i = 1 + 2 + 3 + 4 + \dots + n$$

With Loop:

```
int s=0;
for(int i=1; i<=n; i++){
    s+=i;
}
```

As you can see, in this representation, We are finding sum of all numbers (i's) where i ranges from 1 to n

Example 2:

$$\sum_{i=1}^n 2i = 2 + 4 + 6 + 8 + \dots + 2n$$

With Loop:

```
int s=0;
for(int i=1; i<=n; i++){
    s+=(2*i);
}
```

Example 3:

$$\sum_{i=1}^n 2i - 1 = 1 + 3 + 5 + 7 + \dots + 2n - 1$$

(Sum of all odd numbers)

With Loop:

```
int s=0;
for(int i=1; i<=n; i++){
    s+=(2*i)-1;
}
```

Example 4:

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2$$

With Loop:

```
int s=0;
for(int i=1; i<=n; i++){
    s+=(i*i);
}
```

PI (Pruduct)

** Same goes for PI, but it gives us the product.

$$\prod_{i=1}^n i = 1 * 2 * 3 * 4 * 5 * \dots * n$$

** this is same as $n!$

With Loop:

```
int p=1;
for(int i=1; i<=n; i++){
    p*=i;
}
```

$$\prod_{i=1}^n 2i = 2 * 4 * 6 * 8 * 10 * \dots * 2n$$

With Loop:

```
int p=1;
for(int i=1; i<=n; i++){
    p*=(2*i);
}
```

Mathematical Representation for Sum using Nested loops:

$$\sum_{i=1}^5 \sum_{j=1}^5 j = \underbrace{(1+2+3+4+5)}_{i=1} + \underbrace{(1+2+3+4+5)}_{i=2} + \underbrace{(1+2+3+4+5)}_{i=3} + \underbrace{(1+2+3+4+5)}_{i=4} + \underbrace{(1+2+3+4+5)}_{i=5}$$

**Here, For each iteration of i , j is running from 1 to 5 and adding each value. After every completion of j loop we are getting $(1+2+3+4+5)$ and then adding this value in each iteration of i . As i loop is running for 5 times, We are adding $(1+2+3+4+5)$ this value 5 times.

Finding Mathematical Representation from given series (Nested) and writing loop for its calculation

$$S = \underbrace{(1)}_{\text{term}} + \underbrace{(1+2)}_{\text{term}} + \underbrace{(1+2+3)}_{\text{term}} + \underbrace{(1+2+3+4)}_{\text{term}} + \dots + \underbrace{(1+2+3+4+\dots+n)}_{\text{term}}$$

$ul = \text{upper limit}$
 $ll = \text{lower limit}$

We can see our terms, At First, We will define a general **term**.

Step 1: Look for the sign used in terms. In our case it's plus (+) sign, then we will use **Sigma Σ**

Step 2: Look For constants in term, We can see the lower limit in each term is always **1**

Step 3: Look for variable terms, We can see the upper limit is not fixed, it is ranging from 1 to n , then we will take i for our upper limit. (in our case i is getting incremented by 1)

We got our symbol (Σ), lower limit (**1**) and upper limit (i) then our **term** is $\sum_{j=1}^i j$

We got our term, Now we will find sum of terms, since we have used a variable i as upper limit. We will define i with outerloop. Observe that upper limit (i) is ranging from 1 to n ; then lower limit is **1** and upper limit is n , and there is plus (+) between each term then we will use Sigma Σ

Then our final representation is $\sum_{i=1}^n \sum_{j=1}^i j$

Code:

```
int s=0;
for(int i=1; i<=n; i++){
    int s1=0;
    for(int j=1; j<=i; j++){
        s1+=j;
    }
    s+=s1;
}
System.out.println(s);
```

Another Example:

$$S = (\underset{ll}{1} * \overset{ul(i)}{2}) + (\underset{ll}{1} * 2 * \overset{ul(i)}{3}) + (\underset{ll}{1} * 2 * 3 * \overset{ul(i)}{4}) + (\underset{ll}{1} * 2 * 3 * 4 * \dots * \overset{ul(i)}{n})$$

ul = upper limit
ll = lower limit

In each term (*) product is used. Hence inner loop symbol is (PI) Π

In each term lower limit is fixed at 1.

in our terms upper limit is not fixed. Upper limit is ranging from 2 to n, We take i for it.

Inner loop representation (term): $\prod_{j=1}^i j$

Now, We will define i for outer loop.

Between each term there is plus (+) . then we will use Sigma Σ for it.

i (upper limit of each term) is ranging from 2 to n, then our lower limit for i = 2, and upper limit is n

We got our symbol Σ , lower limit =2, Upper limit =n

Final Representation : $\sum_{i=2}^n \prod_{j=1}^i j$

Code:

```
int s=0;
for(int i=2; i<=n; i++){
    int p=1;
    for(int j=1; j<=i; j++){
        p*=j;
    }
    s+=p;
}
System.out.println(s);
```

Example 2:

$$S = \frac{1}{(1)} + \frac{1}{(1+2)} + \frac{1}{(1+2+3)} + \dots + \frac{1}{(1+2+3+\dots+n)}$$

Representation:
$$\sum_{i=1}^n \frac{1}{\sum_{j=1}^i j}$$

Code:

```
int s=0;
for(int i=1; i<=n; i++){
    int s1=0;
    for(int j=1; j<=i; j++){
        s1+=j;
    }
    int term= 1/s1;
    s=s+term;
}
System.out.println(s);
```

Example 3:

$$S = \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots + \frac{1}{n!}$$

$$[\text{term} = 1/i!] \quad [i! = \prod_{j=1}^i j]$$

Simplifying,
$$S = \frac{1}{1} + \frac{1}{1*2} + \frac{1}{1*2*3} + \dots + \frac{1}{1*2*3*\dots*n}$$

Representation:
$$\sum_{i=1}^n \frac{1}{\prod_{j=1}^i j}$$

Code:

```
int s=0;
for(int i=1; i<=n; i++){
    int p=1;
    for(int j=1; j<=i; j++){
        p*=j;
    }
    int term= 1/p;
    s=s+term;
}
System.out.println(s);
```

Example 4:

$$S = \frac{1}{1} + \frac{1+2}{(1*2)} + \frac{1+2+3}{(1*2*3)} + \dots + \frac{1+2+3+\dots+n}{(1*2*3*\dots*n)}$$

Representation:
$$\sum_{i=1}^n \frac{\sum_{j=1}^i j}{\prod_{j=1}^i j}$$

```
int s=0;
for(int i=1; i<=n; i++){
    int s1=0;
    for(int j=1; j<=i; j++){
        s1+=j;
    }
    int p=1;
    for(int j=1; j<=i; j++){
        p*=j;
    }
    int term= s1/p;
    s=s+term;
}
System.out.println(s);
```